📝 퀴즈 2 — Day 6~14 미리보기 + RAG (50문항, 4지선다)

> `중간점검_Day1-5복습_Day6-14미리보기.md`의 PART B 기반 (특히 B-2 RAG). 먼저 안 보고 풀고 → 맨 뒤 **[답지와 해설]**로 채점 → 틀린 것만 본문 다시 보기. 이 퀴즈에서 6개 이상 맞히면 "RAG 시작 준비 완료"로 봐도 돼.

---

1. Day 6 "통합 테스트"의 목적은?

- ① 새 기능을 추가한다
- ② 처음부터 끝까지 쭉 돌려서 실제로 굴러가는지 눈으로 확인한다
- ③ DB를 삭제한다
- ④ 화면을 만든다

2. Day 6에서 코드를 새로 많이 짜야 하나?

- ① 완전히 새로 짠다
- ② 코드는 새로 안 짜고, 이미 만든 API를 순서대로 눌러본다
- ③ RAG를 짠다
- ④ 서버를 새로 만든다

3. Day 7 README에 들어갈 3가지는?

- ① 로그인, 결제, 배송
- ② API 명세 / ERD / 회고
- ③ 광고, 후기, 가격
- ④ 임베딩, 벡터, 청크

4. ERD가 뜻하는 것은?

- ① AI 모델 이름
- ② 테이블들이 어떻게 연결됐는지 보여주는 관계 그림
- ③ 에러 리포트
- ④ 결제 문서

5. 남은 작업을 3덩어리로 나눈 것으로 맞는 것은?

- ① 로그인 / 결제 / 배포
- ② Day6~7 마무리 / Day8~11 RAG / Day12~14 Streamlit
- ③ 프론트 / 백엔드 / 디자인
- ④ 개발 / 테스트 / 광고

6. RAG는 무엇의 약자로 이해하면 되나?

- ① Random Access Generation
- ② Retrieval(찾아오기) + Augmented(보태기) + Generation(생성)
- ③ Rapid API Gateway
- ④ Read And Give

7. RAG를 한 문장으로?

- ① AI를 두 번 부르는 것
- ② AI에게 답 시키기 전에 관련 참고자료를 찾아서 프롬프트에 끼워주는 것
- ③ DB를 백업하는 것
- ④ 화면을 예쁘게 만드는 것

8. 오픈북 비유에서 "지금 AI(RAG 없음)"는?

- ① 오픈북 시험 학생
- ② 아무것도 안 보고 시험 보는 학생
- ③ 선생님
- ④ 교과서

9. 오픈북 비유에서 "RAG"는?

- ① 관련 페이지를 펴서 보고 답하는 학생
- ② 시험을 안 보는 학생
- ③ 백지 시험
- ④ 감독관

10. 왜 참고자료를 "전부" 프롬프트에 넣으면 안 되나?

- ① 자료가 비밀이라서
- ② 양이 많으면 다 못 넣고, 상관없는 내용까지 섞여 답이 산만해져서
- ③ AI가 화나서
- ④ 법으로 금지돼서

11. "Chunk(청크)"란?

- ① 전체 문서 한 덩어리
- ② 긴 문서를 자른 작은 조각
- ③ 벡터 DB 이름
- ④ AI 모델

12. "Chunking"이란?

- ① 문서를 조각으로 쪼개는 작업
- ② 문서를 합치는 작업
- ③ 문서를 삭제하는 작업
- ④ 문서를 번역하는 작업

13. "Embedding(임베딩)"을 가장 잘 설명한 것은?

- ① 문장을 지도 위의 점(숫자)으로 바꾸는 것
- ② 문장을 삭제하는 것
- ③ 문장을 이미지로 만드는 것
- ④ 문장을 프린트하는 것

14. "Vector(벡터)"란?

- ① 문서 파일 이름

- ② 그 점의 좌표값(숫자 목록)
- ③ AI 모델 종류
- ④ 데이터베이스 이름

15. "Vector DB(벡터 DB)"의 역할은?

- ① 이미지를 저장한다
- ② 점(벡터)들을 저장하고 "가까운 것"을 찾아준다
- ③ 로그인 정보를 담는다
- ④ 화면을 그린다

16. "Retrieval(검색)"이란?

- ① 질문과 가까운 조각을 찾아오는 것
- ② 문서를 쪼개는 것
- ③ 문장을 점으로 바꾸는 것
- ④ DB를 지우는 것

17. 검색해온 조각이 실제로 들어가는 자리 이름은?

- ① child_id
- ② rag_context
- ③ created_at
- ④ response_model

18. 임베딩 "지도 비유"에서 뜻이 비슷한 두 문장은 지도에서?

- ① 서로 멀리 찍힌다
- ② 서로 가까이 찍힌다
- ③ 같은 점에 겹친다
- ④ 지도 밖으로 나간다

19. 컴퓨터가 "칭찬"과 "격려"가 비슷한 말인지 판단하는 방법은?

- ① 사전을 찾아본다
- ② 두 점의 거리가 가까운지 잰다
- ③ 사람에게 물어본다
- ④ 글자 수를 센다

20. 검색이 되는 원리를 한 문장으로 하면?

- ① 질문도 점으로 찍고, 그 점에서 제일 가까운 자료 점들을 집는다
- ② 모든 자료를 무작위로 고른다
- ③ 첫 번째 조각을 항상 쓴다
- ④ 가장 긴 조각을 쓴다

21. RAG의 "준비 단계"는 보통 언제 하나?

- ① 피드백 만들 때마다 매번
- ② 프로그램 켤 때 미리 1번
- ③ 서버 끌 때

- ④ 하루에 100번

22. RAG의 "실행 단계"는 언제 도나?

- ① 프로그램 켤 때만
- ② 피드백을 만들 때마다
- ③ 절대 안 돈다
- ④ 1년에 한 번

23. 준비 단계의 올바른 순서는?

- ① 임베딩 → 문서 준비 → 저장 → 쪼개기
- ② 문서 준비 → 쪼개기(Chunking) → 임베딩 → 벡터 DB 저장
- ③ 저장 → 검색 → 쪼개기 → 문서
- ④ 검색 → 임베딩 → 문서 → 저장

24. 실행 단계의 올바른 순서는?

- ① 질문 임베딩 → 검색 → rag_context에 끼움 → LLM 생성
- ② LLM 생성 → 검색 → 임베딩 → 저장
- ③ 저장 → 쪼개기 → 임베딩 → 검색
- ④ rag_context → 문서 준비 → 저장 → 검색

25. "RAG 없는 지금" 피드백의 특징은?

- ① 근거가 탄탄하고 전문적
- ② 뻔하고 근거가 없다
- ③ 항상 틀린다
- ④ 너무 길다

26. "RAG 붙인 후" 피드백의 특징은?

- ① 참고자료 기반으로 근거와 전문성이 생긴다
- ② 더 짧아지기만 한다
- ③ 랜덤해진다
- ④ 아무 변화 없다

27. 임베딩(점으로 바꾸기)·거리계산은 누가 하나?

- ① 내가 손으로 숫자를 만든다
- ② AI 라이브러리가 다 해준다
- ③ 사용자가 입력한다
- ④ DB가 못 한다

28. Day 9에서 "네가 직접 칠 핵심"에 가까운 것은?

- ① 벡터 DB를 밑바닥부터 구현
- ② `embed()` 부르고 `search()` 불러 결과 꺼내는 줄
- ③ 임베딩 수학 공식 증명
- ④ 서버 인프라 구축

29. RAG 구간에서 "Claude가 깔아줄 것"에 해당하는 것은?

- ① 핵심 개념 이해
- ② 벡터 DB 세팅·연결 보일러플레이트 같은 뼈대
- ③ 자료 내용 고르기
- ④ 검색 결과 눈으로 확인

30. Day 8에서 주로 하는 일은?

- ① 화면 만들기
- ② 참고자료 준비 + 쪼개기(Chunking)
- ③ 결제 연동
- ④ 배포

31. Day 9에서 주로 하는 일은?

- ① 임베딩 + 벡터 저장 + 검색 붙이기
- ② README 작성
- ③ 아이 등록 API
- ④ Streamlit 화면

32. Day 10의 핵심은?

- ① 피드백에 RAG를 연결 (rag_context 자리에 검색 결과 끼우기)
- ② DB 삭제
- ③ 로그인 만들기
- ④ 광고 붙이기

33. `rag_context` 자리는 원래 언제 만들어 뒀나?

- ① Day 8에 처음 만듦
- ② Day 4 `prompt_service.py`에 이미 빈칸으로 만들어 둠
- ③ Day 14
- ④ 만든 적 없음

34. 유리(위축형) 질문에 "줄넘기 손목 스냅" 조각이 안 걸리는 이유는?

- ① 그 조각이 삭제돼서
- ② 질문 점과 거리가 멀어서(상관없어서) 정확히 걸러짐
- ③ 우연히
- ④ AI가 싫어해서

35. Streamlit이 무엇인지 맞는 것은?

- ① 데이터베이스
- ② 파이썬 코드 몇 줄로 웹 화면(버튼·입력창·표)을 만드는 도구
- ③ AI 모델
- ④ 클라우드 서버

36. Streamlit이 필요한 이유는?

- ① Swagger는 개발자용이라 남에게 보여주기 못생겨서, 진짜 앱 화면이 필요해서
- ② 서버가 안 켜져서

- ③ DB가 없어서
- ④ AI가 안 돼서

37. Streamlit 작업의 성격은?

- ① 완전히 새로운 백엔드 로직
- ② 이미 만든 API에 껍데기(화면)를 씌우는 마감 작업
- ③ RAG를 다시 만드는 것
- ④ DB 재설계

38. Streamlit 화면에서 버튼을 누르면 무엇이 호출되나?

- ① 아무것도 안 됨
- ② 우리가 Day 5에 만든 API(예: POST /feedbacks/generate)
- ③ 다른 회사 서버
- ④ 게임 엔진

39. Day 12~14를 순서대로 맞게 나눈 것은?

- ① 12: 배포 / 13: 결제 / 14: 광고
- ② 12: Streamlit 기본 / 13: 화면↔API 연결 / 14: 다듬기·마감
- ③ 12: RAG / 13: DB / 14: 테스트
- ④ 12: 로그인 / 13: 회원가입 / 14: 알림

40. "전체 한 장" 그림에서 남은 일 3가지는?

- ① 로그인 / 결제 / 배포
- ② AI를 똑똑하게(RAG) / 껍데기 씌우기(Streamlit) / 정리(테스트·문서)
- ③ 디자인 / 마케팅 / 영업
- ④ DB / 서버 / 도메인

41. "새 프로젝트가 아니다"라는 안심 포인트의 근거는?

- ① 아무 근거 없음
- ② Day 4에 남긴 rag_context 빈칸을 채우는 것이라서
- ③ 코드를 다 지워서
- ④ 처음부터 다시 해서

42. RAG 시작 전 "말로 설명되면 준비 완료"인 3가지에 포함되지 않는 것은?

- ① RAG를 오픈북 비유로 설명
- ② 임베딩 = 문장을 지도 위 점으로 바꾸는 것
- ③ rag_context 자리에 뭐가 들어가고 왜 필요한지
- ④ Postgres 설치 명령어 외우기

43. `embed("위축형은 과정을 칭찬")`을 부르면 나오는 것은?

- ① 문장의 좌표(벡터/숫자 목록)
- ② 이미지 파일
- ③ 에러
- ④ DB 테이블

44. Chunking(쪼개기)을 하는 이유로 맞는 것은?

- ① 통째론 너무 크고, 필요한 부분만 찾으려고
- ② 문서를 예쁘게 하려고
- ③ 용량을 늘리려고
- ④ 이유 없음

45. 3일 마감 계획에서 "1일차"에 하는 것은?

- ① RAG 전체
- ② Day 6(통합 테스트) + Day 7(README) → 1주차 마무리
- ③ Streamlit 배포
- ④ 결제 연동

46. 3일 계획에서 "가장 무거운 날"은?

- ① 1일차
- ② 2일차 (Day 8~9 RAG 개념 + 쪼개기·임베딩·검색)
- ③ 3일차
- ④ 없음

47. 3일 계획에서 "3일차"에 하는 것은?

- ① Day 10~11(RAG 연결) + Day 12~14(Streamlit) 마감
- ② 아이 등록 API
- ③ DB 재설계
- ④ 회고만

48. "뼈대(구조)는 이미 서 있다"는 말의 뜻은?

- ① Day 1~5로 서버 구조를 이미 완성했다는 뜻
- ② 아직 아무것도 안 했다는 뜻
- ③ 다 지웠다는 뜻
- ④ Streamlit이 끝났다는 뜻

49. RAG 구간에서 임베딩 계산을 "손으로" 해야 하나?

- ① 그렇다, 숫자를 직접 만든다
- ② 아니다, 라이브러리가 하고 나는 흐름·함수 호출만 한다
- ③ 종이에 계산한다
- ④ 엑셀로 한다

50. 다음에 앉으면 할 "시작 문장"으로 맞는 것은?

- ① "일단 유튜브부터 보자"
- ② "의자 앉기 → 화면 열기 → 자가진단 몇 개 말해보기 → Day 6 첫 클릭"
- ③ "내일 하자"
- ④ "처음부터 다시 짜자"

✅ [답지와 해설]

1. ② — 통합 테스트는 부품을 조립해 처음~끝까지 실제로 굴러가는지 확인. 숨은 버그가 여기서 나옴. **2.** ② — 코드 새로 안 짬. 이미 만든 API를 Swagger에서 순서대로 클릭. **3.** ② — README = API 명세 + ERD + 회고. **4.** ② — ERD는 테이블(children─lesson_records─feedbacks) 관계 그림. **5.** ② — 마무리(6~7) / RAG(8~11) / Streamlit(12~14) 3덩어리. **6.** ② — Retrieval + Augmented + Generation. **7.** ② — 답 시키기 전에 관련 참고자료를 찾아 프롬프트에 끼워주는 것. **8.** ② — 지금 AI = 아무것도 안 보고 시험 보는 학생. **9.** ① — RAG = 관련 페이지 펴서 보고 답하는 오픈북 학생. **10.** ② — 양이 많아 다 못 넣고, 상관없는 내용이 섞여 답이 산만해짐. **11.** ② — 청크 = 긴 문서를 자른 작은 조각. **12.** ① — Chunking = 문서를 조각으로 쪼개는 작업. **13.** ① — 임베딩 = 문장을 지도 위 점(숫자)으로 바꾸기. **14.** ② — 벡터 = 그 점의 좌표값(숫자 목록, 예 [0.1,-0.8,...]). **15.** ② — 벡터 DB = 점들을 저장하고 가까운 것을 찾아주는 창고. **16.** ① — Retrieval = 질문과 가까운 조각 찾아오기. **17.** ② — 검색 결과는 rag_context 자리에 들어감. **18.** ② — 뜻이 비슷하면 지도에서 가까이 찍힘. **19.** ② — 두 점의 거리가 가까운지 재서 판단. **20.** ① — 질문도 점으로 찍고 제일 가까운 자료 점을 집음. **21.** ② — 준비 단계는 프로그램 켤 때 미리 1번(문서→쪼개기→임베딩→저장). **22.** ② — 실행 단계는 피드백 만들 때마다 돎(질문 임베딩→검색→끼움→생성). **23.** ② — 문서 준비 → Chunking → Embedding → 벡터 DB 저장. **24.** ① — 질문 임베딩 → 검색 → rag_context에 끼움 → LLM 생성. **25.** ② — RAG 없으면 뻔하고 근거 없음. **26.** ① — RAG 붙이면 참고자료 기반 근거·전문성 생김. **27.** ② — 임베딩·거리계산은 라이브러리가 해줌. 숫자를 손으로 안 만듦. **28.** ② — Day 9 네 몫 = embed()·search() 부르고 결과 꺼내는 줄. **29.** ② — Claude 몫 = 벡터 DB 세팅·연결 같은 보일러플레이트 뼈대. **30.** ② — Day 8 = 자료 준비 + 쪼개기. **31.** ① — Day 9 = 임베딩 + 벡터 저장 + 검색. **32.** ① — Day 10 = rag_context 자리에 검색 결과 끼워 RAG 연결. **33.** ② — rag_context는 Day 4 prompt_service에 이미 빈칸으로 만들어 둠. **34.** ② — 질문 점과 거리가 멀어(상관없어) 정확히 걸러짐 → RAG의 장점. **35.** ② — Streamlit = 파이썬으로 웹 화면 만드는 도구(HTML 몰라도 됨). **36.** ① — Swagger는 개발자용이라, 강사가 쓸 진짜 화면이 필요. **37.** ② — 새 로직 아님. 이미 만든 API에 껍데기 씌우는 마감. **38.** ② — 버튼 → Day 5에 만든 API(POST /feedbacks/generate) 호출. **39.** ② — 12 기본 / 13 화면↔API 연결 / 14 다듬기·마감. **40.** ② — 남은 일: RAG / Streamlit / 정리(테스트·문서). **41.** ② — Day 4에 남긴 rag_context 빈칸을 채우는 것이라 새 프로젝트 아님. **42.** ④ — Postgres 설치 명령 암기는 준비 조건 아님. (a)(b)(c)만 말되면 OK. **43.** ① — embed(문장)은 그 문장의 좌표(벡터)를 돌려줌. **44.** ① — 통째론 크고, 필요한 부분만 찾으려고 쪼갬. **45.** ② — 1일차: Day 6~7로 1주차 마무리. **46.** ② — 2일차 Day 8~9(RAG)가 가장 무거움. **47.** ① — 3일차: Day 10~11 RAG 연결 + Day 12~14 Streamlit 마감. **48.** ① — Day 1~5로 서버 구조(뼈대)를 이미 완성. **49.** ② — 손으로 계산 안 함. 라이브러리가 하고 나는 흐름·호출만. **50.** ② — 시작 문장: 의자 앉기 → 화면 열기 → 자가진단 → Day 6 첫 클릭.